turned on or off, aligning perfectly with the binary system's two-state logic. Additionally, Boolean algebra, the mathematical framework for logical circuit design and operation, enables straightforward implementation of complex operations using simple logic gates with binary inputs: 1 (on / true) or 0 (off / false).

The increased reliability of binary systems stems from their use of only two states. Small variations in signal strength do not affect data integrity as much as in systems using larger bases, making binary more robust in **noisy** environments.

As all data on a computer system is stored in binary, we need systems to represent numerous types of data, such as integers, strings, characters, images, audio and video, in binary form.

- **Noise:** unwanted electrical disturbances that can affect the integrity of signals being processed by a computer; this noise is not related to sound, but to variations in voltage or current that can disrupt the accurate transmission and processing of digital data.
- **Bit:** binary digit; a single digit, either 1 or 0.
- **Byte:** 8 bits.

Representation of integers in binary

To represent numbers in binary, it is useful to remember the basics of our decimal system (base-10).

In the decimal system, as we count, we start with a single digit and increment it by 1 until we reach 9. After 9, we introduce a new digit to the front to represent larger numbers. Let's break down the decimal number 1024:

1000s	100s	10s	1s
1	0	2	4

This can be expressed as:

$(1 \times 1000) + (0 \times 100) + (2 \times 10) + (4 \times 1) = 1024$

Each decimal place value increases by a multiple of 10 as we move to the left because we are working in base-10.

Binary, and other base systems, work in a similar way but, instead of 10 possible states per digit, binary has only two (0 and 1). Consequently, each digit increases by a multiple of 2. Let's break down the binary number 0110:

8s	4s	2s	1s
0	1	1	0

This can be expressed as:

$(0 \times 8) + (1 \times 4) + (1 \times 2) + (0 \times 1) = 6$

In this example, we have no 8s, one 4, one 2 and no 1s. Adding 4 + 2 gives us the decimal (base-10) equivalent of the binary (base-2) number 0110. To clearly denote whether we are showing a binary or decimal number, we usually put the base as a subscript, to avoid confusion:

$0110_2 = 6_{10}$

When working with computer systems, we usually deal with **8-bit binary numbers**. A **bit** can be defined as a "binary digit", and 8 bits is equivalent to 1 **byte**. If the number does not require 8 bits to represent it, we usually pad out the extras with 0s. For example, the decimal number 33 would be represented as:

128s	64s	32s	16s	8s	4s	2s	1s
0	0	1	0	0	0	0	1

This means that with 8 bits, we can represent **256 different numbers**, from **0 to 255**. If we want to represent larger numbers, we need more bits to represent this.

Common mistake

When seeing the number 11111111_2, a common mistake is to say this is 256. However, remember that while there are 256 number possibilities, 255 is the largest number we can represent in 1 byte (as 0 can also be represented).

When referring to bits and bytes, a lowercase "b" is used to represent bits and an uppercase "B" is used to represent bytes. We then use a prefix system as the numbers increase.

There are two types of prefixes when referring to bits and bytes: one for base-10 (e.g. kilo, mega, giga) and another for base-2. There was a time when base-10 prefixes were also used for base-2 quantities due to their similarity (e.g. 1024 is close to 1000). However, the confusion this generated led to calls for change. To address this, in 1999 the IEC introduced new prefixes (e.g. kibi, mebi, gibi) specifically for base-2 multiples (1024, 1,048,576, 1,073,741,824).

Kibibyte KiB	**Mebibyte MiB**	**Gibibyte GiB**	**Tebibyte TiB**	**Pebibyte PiB**	**Exbibyte EiB**	**Zebibyte ZiB**
1 KiB = 1024 bytes	1 MiB = 1024 KiB	1 GiB = 1024 MiB	1 TiB = 1024 GiB	1 PiB = 1024 TiB	1EiB = 1024 PiB	1 ZiB = 1024 EiB
Kilobyte KB	**Megabyte MB**	**Gigabyte GB**	**Terabyte TB**	**Petabyte PB**	**Exabyte EB**	**Zettabyte ZB**
1 KB = 1000 bytes	1 MB = 1000 KB	1 GB = 1000 MB	1 TB = 1000 GB	1 PB = 1000 TB	1 EB = 1000 PB	1 ZB = 1000 EB

REVIEW QUESTION

Bits and byte notation are worth knowing when dealing with mobile-phone and internet companies.

Download speeds of up to 100Mb/s!

or

Download speeds of up to 100MB/s!

If the two advertisements above were from two different internet companies, assuming the cost is the same, which one offers faster speeds and by how much?

Converting binary numbers to decimal

There are two main methods for converting a binary number to decimal: the positional notation method and the doubling method.

Positional notation method:

This is possibly the most straightforward method, where you assign the place values and sum.

1. Starting from the right, assign the place values for each binary bit.
2. Sum each of the place values that has a 1 underneath it.

For example, to convert 10111011_2 to decimal:

128	**64**	**32**	**16**	**8**	**4**	**2**	**1**
1	0	1	1	1	0	1	1

128 + 32 + 16 + 8 + 2 + 1 = 187

Doubling method:

1. Start with the leftmost bit (the most significant bit).
2. Double the current total and add the next bit.
3. Repeat until all bits are processed.

Top tip!

When converting to and from binary, it is always a good idea to write the digit place values down first. Trying to remember these in your head can lead to silly mistakes.

128 64 32
16 8 4
2 1

For example, to convert 10111011_2 to decimal:

Step	Binary digit	Current total	Calculation
1	1	1	Initial value
2	0	2	1 × 2 + 0 = 2
3	1	5	2 × 2 + 1 = 5
4	1	11	5 × 2 + 1 = 11
5	1	23	11 × 2 + 1 = 23
6	0	46	23 × 2 + 0 = 46
7	1	93	46 × 2 + 1 = 93
8	1	187	93 × 2 + 1 = 187

Common mistake

If you use this method, remember to start with the most significant bit (MSB), not the **least significant bit** (LSB).

◆ **Least significant bit (LSB):** the rightmost bit in a binary number, representing the smallest value position (0 or 1).

◆ **Quotient:** the result obtained when one number is divided by another, e.g. in the division of 15 by 3, the quotient is 5.

REVIEW QUESTIONS

Convert the following binary (base-2) numbers to decimal (base-10):

1 11001010_2
2 01101101_2
3 10110011_2
4 00011110_2
5 11100001_2

Converting decimal numbers to binary

There are two main methods for converting a decimal number to binary: the division method and the subtraction method.

Division method:

1 Divide the decimal number by 2.
2 Write down the **quotient** and the remainder.
The remainder will be either 0 or 1. This represents a digit of the binary number (the LSB on the first division).
3 Update the quotient.
4 Repeat until the quotient is 0.
5 Construct the binary number (this is read from the remainders from the first to the last).

For example, to convert 42_{10} to binary:

Division step	Quotient	Remainder
42 / 2	21	0
21 / 2	10	1
10 / 2	5	0
5 / 2	2	1
2 / 2	1	0
1 / 2	0	1

Construct the binary number from the remainders and pad to 8-bits: 00101010_2

Common mistake

Remember to construct the remainders in the correct order to format your binary number. The first remainder is the least significant bit (LSB).

Subtraction method:

Write down the place values for an 8-bit binary number:

128 64 32 16 8 4 2 1

Starting with the largest place value (128):

1. Try and subtract it from the number you are converting.
   - ☐ If the place value is larger than the number, write a 0 below it.
   - ☐ If it is smaller or equal to it, write a 1 and calculate the remainder of the subtraction, carrying the result to the next place value.
2. Repeat.

For example, to convert 42_{10} to binary:

128_{10} and 64_{10} are larger than 42_{10}, so we write 0 below these.

128	64	32	16	8	4	2	1
0	0						

32_{10} is smaller, so we write a 1 below it and calculate the remainder from the subtraction, the result of which will carry to the next place value:

$42_{10} - 32_{10} = 10_{10}$

128	64	32	16	8	4	2	1
0	0	1	0				

16 is larger than 10, so write a 0

8 is smaller, so write a 1 and calculate the remainder:

$10_{10} - 8_{10} = 2_{10}$

4_{10} is larger than 2_{10}, so write a 0

2_{10} is equal, so calculate the remainder (0) and write a 1

128	64	32	16	8	4	2	1
0	0	1	0	1	0	1	0

REVIEW QUESTIONS

Convert the following decimal (base-10) numbers to binary (base-2):

1. 20_{10}
2. 87_{10}
3. 123_{10}
4. 199_{10}
5. 250_{10}

PROGRAMMING EXERCISE

Write a binary-to-decimal and decimal-to-binary application in either Python or Java.

Representation of integers in hexadecimal

Hexadecimal (often abbreviated as hex) is a base-16 number system that uses 16 distinct symbols to represent values, rather than the 10 of decimal or 2 of binary. The symbols include the digits 0 to 9 and then the letters A to F, where A represents 10, B represents 11, C represents 12, D represents 13, E represents 14 and F represents 15.

◆ **Debugging tools:** software applications or utilities used by developers to identify, analyse and fix bugs or issues within a program by inspecting code, variables and execution flow.

◆ **Memory dump:** a process where the contents of a computer's memory are captured and saved, typically for the purpose of diagnosing and debugging software issues.

◆ **Nibble:** 4 bits.

Hexadecimal is used with computers for several reasons. The ease of conversion between binary and hexadecimal is straightforward because each hex digit maps directly to a 4-bit binary sequence. For example, the binary number 1111 can be represented as F in hex. Another reason is that it provides a more compact way to represent a binary value. This makes it much easier for us to read and communicate large binary numbers. This is why you often see hex used in **debugging tools**, **memory dumps** and assembly language programming.

Converting binary numbers to hexadecimal

Converting binary to hexadecimal is a straightforward calculation.

1. Split the binary byte (8 bits) into two **nibbles** (2 × 4 bits).
2. Calculate the decimal value of these 4 bits.
3. Convert the decimal values into their hexadecimal equivalents and rejoin them.

For example, to convert 01101011_2 to hexadecimal:

1. Split the byte into 2 nibbles:
   0110_2 1011_2
2. Calculate the decimal value:
   6_{10} 11_{10}
3. Convert both decimal values to their hexadecimal equivalents and rejoin them:
   $6B_{16}$

REVIEW QUESTIONS

Convert the following binary (base-2) numbers to hexadecimal (base-16):

1. 10101100_2
2. 11010110_2
3. 11010001_2
4. 00111010_2
5. 10011101_2

Converting hexadecimal numbers to binary

Moving from hex to binary is just a reverse of the binary-to-hexadecimal process:

1. Split the two hexadecimal digits.
2. Convert each of them to a 4-bit binary number using the same integer-to-binary method.
3. Join the two 4-bit numbers together to form 1 byte.

For example, to convert $F2_{16}$ to binary:

1. Split the two hexadecimal digits:
   F_{16} 2_{16}
2. Convert each of them to a 4-bit binary number using the same integer-to-binary method:
   1111_2 0010_2
3. Join the two 4-bit numbers together to form 1 byte:
   11110010_2

REVIEW QUESTIONS

Convert the following hexadecimal (base-16) numbers to binary (base-2):

1 $3F_{16}$
2 $A9_{16}$
3 10_{16}
4 $7C_{16}$
5 $E2_{16}$

Converting decimal numbers to hexadecimal

To move between decimal and hexadecimal is one of the trickier calculations to perform, as you need to be comfortable with your 16 times table. To convert a decimal number to hex:

1 Divide the decimal number by 16 and record the remainder.
2 Repeat the process with the quotient until the quotient is 0.
3 Form the hex number from the remainders, with the last remainder obtained being the most significant bit (the number on the left).

For example, to convert 254_{10} to hexadecimal:

1 Divide the decimal number by 16 and record the remainder:
   $254_{10} / 16_{10} = 15_{10}$ remainder 14_{10}
   quotient = 15_{10}
   remainder $14_{10} = E_{16}$
2 Repeat the process with the quotient until the quotient is 0:
   $15_{10} / 16_{10} = 0_{10}$ remainder 15_{10}
   quotient = 0_{10}
   remainder $15_{10} = F_{16}$
3 Form the hex number from the remainders, with the last remainder obtained being the most significant bit (the number on the left):
   FE_{16}

REVIEW QUESTIONS

Convert the following decimal (base-10) numbers to hexadecimal (base-16):

1 42_{10}
2 157_{10}
3 89_{10}
4 200_{10}
5 123_{10}

Converting hexadecimal numbers to decimal

To convert from hexadecimal to decimal:

1 Convert hex digits to their decimal equivalents.
2 Multiply them by 16 raised to the power of its position index, starting from 0 on the right.
3 Sum the results.

For example, to convert $2F_{16}$ to decimal:

1 Convert hex digits to their decimal equivalents:
   $2_{16} = 2_{10}$
   $F_{16} = 15_{10}$

2 Multiply them by 16 raised to the power of its position index, starting from 0 on the right:
$2 \times 16^1 = 2 \times 16 = 32_{10}$
$15 \times 16^0 = 15 \times 1 = 15_{10}$
3 Sum the results:
$32 + 15 = 47_{10}$

PROGRAMMING EXERCISE

Add hexadecimal conversion functionality to the binary converter app you created before.

A1.2.2 How b inary is used to store data

The binary system underpins everything from numerical values and textual information to complex multimedia files, ensuring efficient and reliable data processing. In this section, we are going to discover the mechanisms that are used to store such data as characters, strings, images, audio and video in binary form.

Characters and strings

Characters and strings are stored using standardized binary encoding schemes, enabling consistent storage, retrieval and processing across different systems and applications. The most common encoding standards are ASCII (American Standard Code for Information Interchange) and Unicode.

ASCII encoding

The development of ASCII began in 1960 and was officially standardized in 1963. It was developed because there was no standardized way to encode text characters, which led to compatibility issues between devices and systems. Each manufacturer used its own proprietary encoding system, which made it very difficult for devices to communicate with each other. ASCII was designed to provide a common standard for the interchange of text data.

ASCII initially started out as a 7-bit encoding system, which gave it the ability to represent 128 (2^7) different characters, which was considered sufficient for most basic text data (letters, numbers, punctuation and control characters). However, as computing became more global and applications required support for additional characters, an 8-bit extension to ASCII was developed, giving it the ability to represent 256 (2^8) characters. This was referred to as extended ASCII, and the new characters were mainly used for Western European languages.

ASCII uses a simple but clever system to represent characters in binary (as long as we are only considering the Latin (English) alphabet). The first five bits from the right are used to represent the letter by its numerical place in the alphabet.

For example:

01100001_2 = 1_{10} = a
01100010_2 = 2_{10} = b
01100011_2 = 3_{10} = c

Top tip!

If the first five bits from the right are 00000 (five zeros), it is almost certainly a space (00100000).

If the first three bits from the left are not 011 or 010, it is likely to be a punctuation mark.

The first three bits from the left represent whether it is an uppercase or lowercase letter. 011 = lowercase; 010 = uppercase:

01100001_2 = a

01000001_2 = A

REVIEW QUESTION

Convert the following binary back into text to reveal the hidden message.

01000110 01101111 01101100 01101100 01101111 01110111 00100000 01110100 01101000 01100101 00100000 01110111 01101000 01101001 01110100 01100101 00100000 01110010 01100001 01100010 01100010 01101001 01110100

PROGRAMMING EXERCISE

Create an application so that you can send secret messages to your friends.

Write an application that accepts either a string of characters or a stream of binary. It should either encode the characters using ASCII and binary or convert the binary back into text.

To make the binary less easy to decode by hand, you could remove all spacing between the 8-bit characters.

Unicode encoding

In the 1960s, the United States and the majority of English-speaking countries had a system in 7-bit ASCII that worked for the English alphabet. Other non-English speaking countries had their own unique encoding systems to work with their own languages. When the ASCII system was increased to 8 bits (extended ASCII), allowing for 256 characters for use in modern computers, countries did not agree on the same standard. Nordic countries started using the extra space to encode characters for their own languages, and Japan used four different systems that were not even compatible with each other. This was not a huge issue as communication between these systems was rare, but then the internet was launched and compatibility became very important as more and more information was being shared between systems in different countries.

In 1991, the Unicode Consortium was created to try and solve this problem. The organization was established to develop, maintain and promote the Unicode Standard, which provides a unique number for every character, regardless of platform, program or language. It needed to create a system that was capable of storing all the characters and punctuation marks from all the languages in the world, but also wanted it to be backwards compatible with ASCII. At the time of writing, the current Unicode Standard version 15.0, released in September 2022, encodes 149,186 different characters. Unicode includes the Latin, Cyrillic, Greek and Arabic alphabets, and Chinese characters, as well as many others, and also includes emojis and mathematical and other technical symbols. In Unicode, each letter or symbol is assigned a unique number, for example:

- A = 65
- 汉 = 27721
- 💩 = 128169

You can find the numerical representation for any character or symbol using the code below:

Python

```python
# Python examples
char_a = 'A'
char_han = '汉'
char_poo = '💩'
# Get Unicode code points as integers
code_point_a = ord(char_a)  # 65
code_point_han = ord(char_han)  # 27721
code_point_poo = ord(char_poo)  # 128169
# Print integer representations
print(code_point_a)  # Output: 65
print(code_point_han)  # Output: 27721
print(code_point_poo)  # Output: 128169
```

Java

```java
public class UnicodeExample {
    public static void main(String[] args) {
        // Define characters
        char charA = 'A';
        char charHan = '汉';
        String charPoo = "💩"; // Note: Java uses UTF-16 and
        // the emoji is usually a surrogate pair
        // Get Unicode code points as integers
        int codePointA = (int) charA; // 65
        int codePointHan = (int) charHan; // 27721
        int codePointPoo = charPoo.codePointAt(0); // 128169
        // Print integer representations
        System.out.println("Unicode code point of 'A': " +
        codePointA); // Output: 65
        System.out.println("Unicode code point of '汉': " +
        codePointHan); // Output: 27721
        System.out.println("Unicode code point of '💩': " +
        codePointPoo); // Output: 128169
    }
}
```

How did they manage this? The story is that it was conceived in a café on the back of a napkin when Joe Becker (Xerox), Lee Collins (Apple) and Mark Davis (Apple and later Google) met and designed the encoding scheme in 1987. There are a few different versions of Unicode: UTF-8, UTF-16 and UTF-32. Each has its own uses:

	UTF-8	UTF-16	UTF-32
Variable length encoding	1–4 bytes per character	2 or 4 bytes per character	4 bytes per character
Note	Compatibility: backward compatible with ASCII	Surrogate pairs: for characters outside the **Basic Multilingual Plane (BMP)**, two 16-bit code units are used	Simplicity: easier to process because each character is exactly 4 bytes
Usage	Most commonly used encoding on the web and in many applications	Often used in Windows and Java environments	Less common due to higher storage requirements

◆ **Basic Multilingual Plane (BMP):** the most commonly used characters and symbols for almost all modern languages.

Let's examine UTF-8, the most commonly used encoding system, and understand its functionality.

Instead of merely expanding the size to accommodate over 100,000 characters, which would have adversely impacted most online content, a more efficient solution was devised. Had all characters been standardized to use 32 bits, each letter in the ASCII system would have quadrupled in size. This would have resulted in significantly larger documents and web pages, leading to increased storage requirements and slower transfer times. The system also needed never to send eight zeros (00000000) in a row, as many older systems would see this as the end of communication and would stop listening.

So the UTF-8 system kept the ASCII system the same. The letter "A" is encoded as:

01000001 = A

However, if the character needed went beyond the standard ASCII system, "é" for example, more than one byte would be required:

11000011 **10**101001 = é

The bits in bold are important. The first three significant bits "110" on the first byte represent that this character is made up of two bytes in total (a 0 is needed at the end to show when this information is finished). The second byte starts "10", which means this is a continuation. If you remove those 5 bits and then put both bytes together:

000 1110 1001 = 233 = é

Another example is:

11110000 **10**011111 **10**011000 **10**000100 = 😩

This emoji requires four bytes using the UTF-8 system. The first byte communicates that this character is made up of four bytes ("11110") and the next three bytes start with "10", showing they are continuation bytes. If we remove that information:

0001 1111 0110 0000 0100 = 128516 = 😩

UTF-8 has been adopted by the internet as the main character encoding system; however, it doesn't come without some issues. Due to the variable length, some characters (especially those from Asian languages or emojis) take more space compared to single-byte encodings. This can lead to larger file sizes in certain contexts. The processing required to handle variable-length encoding also requires more complex processing compared to fixed-length systems such as UTF-32.

◆ **Shift cipher:** a type of substitution cipher, where each letter in the plaintext is shifted a certain number of positions down or up the alphabet.

Despite these issues, UTF-8 has proved to be a versatile and effective encoding standard that meets the needs of the modern internet. Its backward compatibility, efficiency and broad support make it an enduring choice for encoding text. While it does have some challenges, particularly with handling non-ASCII characters and variable-length encoding, these are not significant enough ever to warrant a wholesale replacement. Therefore, it's likely that UTF-8 will continue to be the dominant text encoding standard for the foreseeable future.

PROGRAMMING EXERCISES

The code below uses a Caesar cipher to encrypt the string that is input using a key. A Caesar cipher is a simple **shift cipher**, where each letter is considered to be an integer (a = 1, b = 2, c = 3, and so on) and the key is added to this to find the encrypted letter, for example:

String input: "Hello"

Key input: 1

Output: Ifmmp

Python

```
def caesar_cipher_encrypt(message, key):
    encrypted_message = ""
    for char in message:
        if char.isalpha():  # Check whether the character is a letter
            shift = ord("A") if char.isupper() else ord("a")  # Determine the
            # ASCII offset
            # Shift the character and wrap around the alphabet if necessary
            encrypted_char = chr((ord(char) - shift + key) % 26 + shift)
            encrypted_message += encrypted_char
        else:
            encrypted_message += char  # Non-letter characters remain unchanged
    return encrypted_message
# User input
message = input("Enter the message to encrypt: ")
key = int(input("Enter the key (an integer): "))
# Encrypt the message
encrypted_message = caesar_cipher_encrypt(message, key)
print(f"Encrypted message: {encrypted_message}")
```

Java

```
import java.util.Scanner;
public class CaesarCipher {
    public static String caesarCipherEncrypt(String message, int key) {
        StringBuilder encryptedMessage = new StringBuilder();
        for (char ch : message.toCharArray()) {
            if (Character.isLetter(ch)) {  // Check whether the character is
            // a letter
                char shift;
                if (Character.isUpperCase(ch)) {
                    shift = 'A';  // Determine the ASCII offset for uppercase
                    // letters
                } else {
                    shift = 'a';  // Determine the ASCII offset for lowercase
                    // letters
                }
                // Shift the character and wrap around the alphabet if
                // necessary
                char encryptedChar = (char) ((ch - shift + key) % 26 + shift);
                encryptedMessage.append(encryptedChar);
            } else {
                encryptedMessage.append(ch);  // Non-letter characters remain
                // unchanged
            }
        }
        return encryptedMessage.toString();
    }
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        // User input
        System.out.print("Enter the message to encrypt: ");
        String message = scanner.nextLine();
        System.out.print("Enter the key (an integer): ");
        int key = scanner.nextInt();
        // Encrypt the message
        String encryptedMessage = caesarCipherEncrypt(message, key);
        System.out.println("Encrypted message: " + encryptedMessage);
    }
}
```

1 After studying how this code works, write the decrypt function for someone who receives an encrypted message.

2 Write a function that is able to **brute force** an encrypted message so you can identify the key used.

◆ **Brute force:** a method of breaking a cipher by systematically trying every possible key until the correct one is found.

The first ever digital image: Russel Kirch's son, Walden, in 1957

Images

In 1957, Russel Kirch scanned an **analogue** photo of his son Walden, converting the picture into a digital file. This was the first ever digital image created. It was a significant milestone in the evolution of visual technology, revolutionizing the way we capture, store and manipulate pictures. The development of early digital cameras and scanners, which enabled devices to convert light into digital data, started the trend that has now become commonplace, and the transition from film to digital has transformed numerous industries, from photography and medical imaging to telecommunications and entertainment.

Bitmap images

Bitmap images, also known as "raster" images, are one of the most fundamental forms of digital graphics. They reproduce images by using a grid of **pixels**, with each pixel assigned a specific colour and intensity.

At the bottom of the page is a bitmap image with an **image resolution** dimension of 13×10 (13 pixels wide by 10 pixels high). Each pixel is "described" using 1 bit of data: either 1 or 0. In this case, 1 = black and 0 = white (a monochrome image), and the amount of bits used to describe the colour is known as the "bit depth" or **colour depth**. So, we have a 13×10 image with a 1-bit colour depth in this example.

To calculate the size of this image, the formula is:

image size = width (pixels) × height (pixels) × colour depth (bits per pixel)

13 × 10 × 1 = 130 bits (or 130 / 8 = 16.25 bytes)

However, in reality, this calculation is not completely accurate, as the image would require more data to store **metadata** and other header information. This could include information such as dimensions, colour depth and other attributes that allow the CPU to read the image data accurately so it displays the image correctly to the screen.

0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	0	1	0	0	0
0	0	0	1	0	0	0	0	1	0	0	0	0
0	0	1	1	1	1	1	1	1	1	1	0	0
0	0	1	1	0	1	1	1	0	1	1	0	0
0	1	1	1	1	1	1	1	1	1	1	1	0
0	1	0	1	0	0	0	0	0	1	0	1	0
0	1	0	1	0	0	0	0	0	1	0	1	0
0	0	0	0	1	1	0	1	1	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0

◆ **Analogue:** a continuous signal that represents varying physical quantities, such as sound waves, which varies smoothly over a range; digital represents data in discrete binary values (0s and 1s), enabling precise and error-resistant processing.

◆ **Bitmap:** a type of digital image composed of a grid of pixels, each holding a specific colour value, representing the image in a rasterized format.

◆ **Pixel:** short for "picture element"; the smallest unit of a digital image or display, representing a single point in the image with a specific colour and intensity.

◆ **Image resolution:** the number of pixels contained within a digital image, typically expressed as the dimensions (width × height) in pixels, and sometimes as the pixel density (PPI / DPI) for print quality.

◆ **Colour depth:** also known as "bit depth"; the number of bits used to represent the colour of each pixel in a digital image, determining the range and precision of colours that can be displayed.

◆ **Metadata:** information that describes other data, providing context and details about the data's content, structure and attributes. In the context of digital images, metadata includes such information as the image's dimensions, colour depth, creation date, author, camera settings and other properties that help with managing, understanding and using the image effectively.

To improve the quality of a bitmap image, we have two options: We can increase the number of pixels (resolution) or we can increase the colour depth.

Resolution – increasing the number of pixels:

Increasing the number of pixels in a bitmap image increases the image quality. A higher resolution allows for greater detail and clarity, and images with lower resolutions can lead to a loss of detail and a pixelated appearance. However, the quantity of pixels is not the only consideration: the size of the screen they are displayed on is also important. Images with a higher PPI (pixels per inch) look clearer than those with a lower PPI. Imagine having an image with a resolution of 1024×768 shown on your phone compared to on a cinema screen. The higher PPI on the phone will give a clearer image due to the increased pixel density. The trade-off for a higher resolution image is larger file size, which can impact storage and transfer efficiency.

■ Common image resolutions

Resolution name	Pixel dimensions	Common usage
VGA	640 × 480	Early computer screens, basic web graphics
SVGA	800 × 600	Standard computer monitors, web graphics
HD (720p)	1280 × 760	HD video, basic HD television
Full HD (1080p)	1920 × 1080	Full HD video, modern monitors and televisions
2K	2048 × 1080	Digital cinema, some monitors
Quad HD (1440p)	2560 × 1440	High-resolution monitors, gaming, professional use
4K (Ultra HD)	3840 × 2160	Ultra HD televisions, high-end monitors, video
8K	7680 × 4320	Cutting-edge televisions, professional video

Colour depth – increasing the amount of colours:

When we increase the colour depth, it allows for a wider range of colours to be represented, resulting in more vibrant and accurate images. If an image's colour depth is low, this can lead to banding, where gradients appear as distinct steps rather than smooth transitions. However, just like image resolution, we must also consider the impact of file size for storage and transfer times. The higher the colour depth, the larger the file size.

To work out the number of colours available, we calculate 2 to the power of the colour depth of the image; for example, for an image with an 8-bit colour depth:

$2^8 = 256$

■ Common colour depths

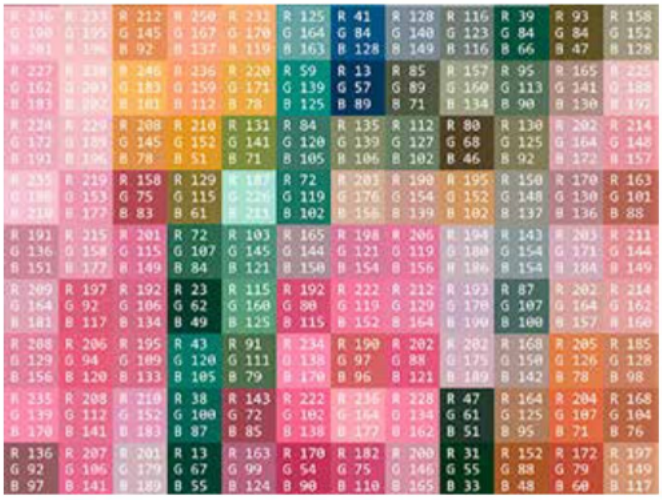
Colour depth (bits per pixel)	Number of colours	Common usage
1 bit	2	Simple graphics, monochrome displays
4 bit	16	Early computer graphics, icons
8 bit	256	GIF images, simple web graphics
16 bit	65,536	High-colour images, some video formats
24 bit (true colour)	16.8 million	Standard for most images and video, digital photography
30 bit (deep colour)	Over 1 billion	Professional photography, high-end monitors and televisions
36 bit	Over 68 billion	Medical imaging, professional graphics
48 bit	Trillions	High-end personal applications, detailed scientific imaging

■ A high-resolution image with a resolution of 2268 × 4032, a 24-bit colour depth and a file size of 1.77 MB

The majority of modern-day screens are 24 bit, allowing for 16.8 million colours. They have three lights per pixel: a red, a green and a blue light, otherwise known as "RGB", and have a value range from 0 to 255 (1 byte per colour channel). This is sufficient for most applications, as most human eyes can only distinguish between around 10 million distinct colours. Monitors that go beyond 24 bit are normally only necessary in professional fields where precision is crucial.

On the left is a high-resolution image. If we zoom in to the dress on this image, we can see the breakdown of the individual pixels and the values of the distinct colour channels. When working with graphics, these values are often shown in hexadecimal. If we take the top left pixel of the dress as an example:

R: 216, G: 190, B: 199 = #d8bec7



■ A zoomed-in area of the image above, showing the value of each pixel – created using www.csfieldguide.org.nz/en/interactives/pixel-viewer

■ The colour values for the top left pixel of the dress in the photo – created using www.w3schools.com/colors/colors_rgb.asp

We can also see the impact of lower colour depths on the same image:

■ The same image using multiple colour depths: 24 bits to 0 bits – created using www.csfieldguide.org.nz/en/interactives/image-bit-comparer

REVIEW QUESTIONS

1 A bitmap image uses a colour depth of 3 bits, allowing for eight distinct colours.

How many bits are needed to represent the colours if the bitmap image uses 32 distinct colours?

2 Raj is creating a bitmap graphic for a game. The image dimensions are 10 pixels wide and 12 pixels tall.

How many pixels are there in total in the image?

3 Alice is organizing her digital artwork collection that she has created over the years.

While transferring her artwork files to a new cloud storage service, she notices that each file is larger than she anticipated. This is because, aside from the actual image data, the file includes extra information necessary for accurate reproduction of the image. What is this additional information, which contains details about the pixel data, called?

4 Determine the storage capacity needed for a bitmap image with dimensions of 800 × 600 pixels that supports 512 different colours.

Then, calculate the file size in kilobytes (kB) if the file metadata occupies an additional 25 per cent of the space. Present your answer as a real number, including the decimal values.

PROGRAMMING EXERCISES

Here are some fun ways to explore images in more depth using Python or Java.

1 Extract and print RGB values.

Java

```
import java.awt.Color;
import java.awt.image.BufferedImage;
import java.io.File;
import java.io.IOException;
import javax.imageio.ImageIO;
public class ImageToRGB {
    public static void main(String[] args) {
        try {
            // Load the image
            BufferedImage image = ImageIO.read(new File("sample_image.jpg"));
            // Get image dimensions
            int width = image.getWidth();
            int height = image.getHeight();
            // Loop through each pixel
            for (int y = 0; y < height; y++) {
                for (int x = 0; x < width; x++) {
                    // Get the RGB value of the pixel
                    int pixel = image.getRGB(x, y);
                    Color color = new Color(pixel);
                    // Extract the red, green and blue components
                    int red = color.getRed();
                    int green = color.getGreen();
                    int blue = color.getBlue();
                    // Print the RGB values
                    System.out.println("Pixel at (" + x + ", " + y + "): R=" +
                    red + ", G=" + green + ", B=" + blue);
                }
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

For Python, you need to install the Pillow library first. Run this command in your terminal to install the necessary libraries:

```
pip install pillow
```

Python

```
pip install pillow
Use this code to access the RGB values for each pixel
from PIL import Image
# Load the image
image = Image.open("sample_image.jpg")
# Convert the image to RGB mode
image = image.convert("RGB")
# Get the image dimensions
width, height = image.size
# Extract and print RGB values
for y in range(height):
    for x in range(width):
        pixel = image.getpixel((x, y))
        red, green, blue = pixel
        print(f"Pixel at ({x}, {y}): R={red}, G={green}, B={blue}")
```

2 Apply a grayscale filter.

Warning:

This code processes the image pixel by pixel, which means it iterates through every pixel in the image to apply the grayscale filter. For very large images (e.g. high-resolution photos), this process can be computationally intensive and take a significant amount of time to complete. Consider testing this code on smaller images first (e.g. 100x100 pixels) to observe its behaviour before applying it to larger files.

Python

```
from PIL import Image
# Load the image
image = Image.open("sample_image.jpg")
# Convert the image to RGB mode
image = image.convert('RGB')
# Get the image dimensions
width, height = image.size
# Create a new image to store the grayscale result
grayscale_image = Image.new("RGB", (width, height))
# Apply a grayscale filter
for y in range(height):
    for x in range(width):
        pixel = image.getpixel((x, y))
        red, green, blue = pixel
        grayscale = int(0.3 * red + 0.59 * green + 0.11 * blue)
        grayscale_image.putpixel((x, y), (grayscale, grayscale, grayscale))
# Save the grayscale image
grayscale_image.save("grayscale_image.jpg")
```

Java

```
import java.awt.Color;
import java.awt.image.BufferedImage;
import java.io.File;
import java.io.IOException;
import javax.imageio.ImageIO;
public class ImageToGrayScale {
    public static void main(String[] args) {
        try {
            // Load the image
            BufferedImage image = ImageIO.read(new File("sample_image.jpg"));
            // Get image dimensions
            int width = image.getWidth();
            int height = image.getHeight();
            // Create a new image to store the grayscale result
            BufferedImage grayscaleImage = new BufferedImage(width, height,
            BufferedImage.TYPE_INT_RGB);
            // Apply a grayscale filter
            for (int y = 0; y < height; y++) {
                for (int x = 0; x < width; x++) {
                    // Get the RGB value of the pixel
                    int pixel = image.getRGB(x, y);
                    Color color = new Color(pixel);
                    // Extract the red, green and blue components
                    int red = color.getRed();
                    int green = color.getGreen();
                    int blue = color.getBlue();
                    // Compute the grayscale value
                    int grayscale = (int) (0.3 * red + 0.59 * green + 0.11 *
                    blue);
                    // Create a new Color object with the grayscale value
                    Color grayColor = new Color(grayscale, grayscale, grayscale);
                    // Set the new pixel value in the grayscale image
                    grayscaleImage.setRGB(x, y, grayColor.getRGB());
                }
            }
            // Save the grayscale image
            ImageIO.write(grayscaleImage, "jpg", new File("grayscale_image_
            java.jpg"));
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

3 After studying how the grayscale filter works, are you now able to create your own unique filters?

Audio

Audio in its analogue form is a continuous signal that represents sound waves through variations of air pressure. These sound waves can be captured through input devices, such as microphones, which convert the sound waves into a digital signal, which is stored as binary. This process involves several steps:

◆ **Amplitude:** the magnitude of change in a sound wave, representing the loudness or intensity of the sound.

◆ **Sampling:** the process of converting a continuous analogue signal into a series of discrete digital values by measuring the signal's amplitude at regular intervals.

◆ **kHz (kilohertz):** a unit of frequency equal to 1000 cycles per second, commonly used to measure the sampling rate of audio signals.

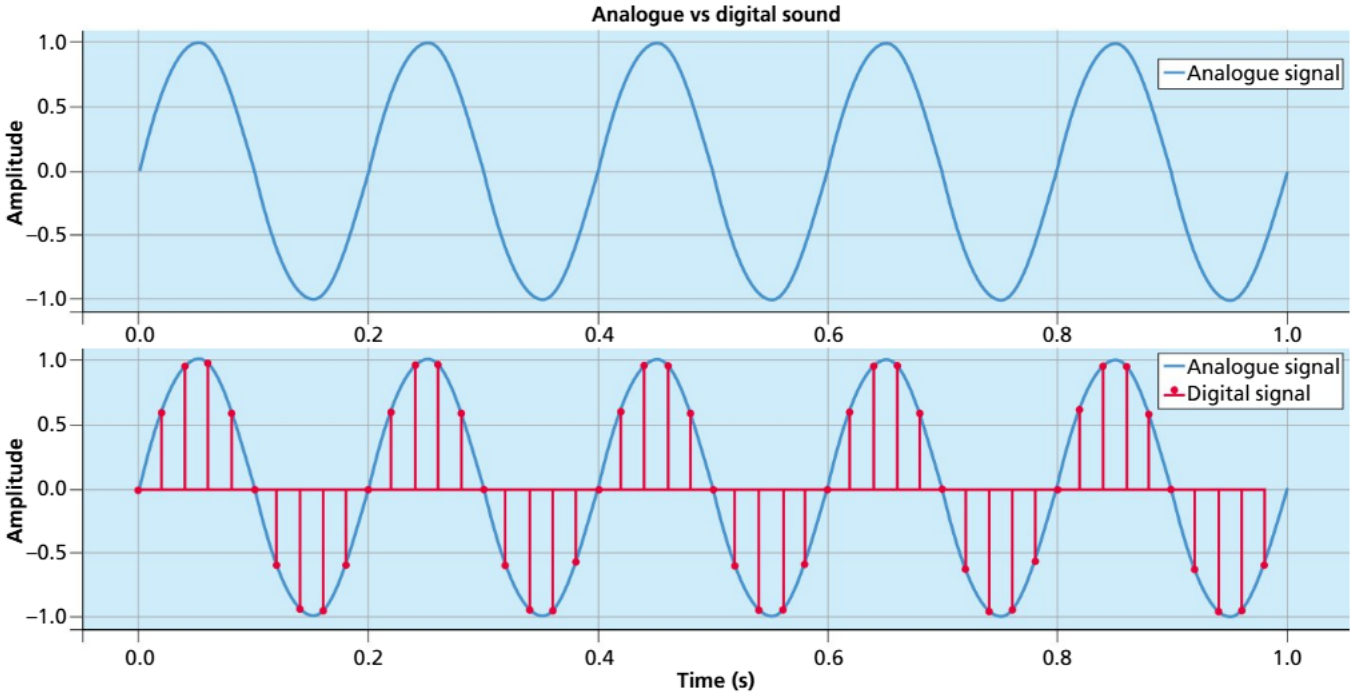
Analogue-to-digital conversion (ADC)

Sound is a continuous analogue signal. An ADC samples the **amplitude** (loudness) of the sound at discrete intervals in a process known as **sampling**. The rate at which this happens is measured in Hertz (Hz) – the higher the Hertz, the more samples are recorded per second. CD-quality audio uses 44.1 **kHz**, but professional quality audio is sampled at 48 kHz.

This sample is then stored and represented as a numerical value in binary. The precision is determined by the bit depth. The larger the bit depth, the more possible values that can be used to describe the sample. For example, the bit depth of CD-quality sound is 16 bit, which gives 2^{16}, or 65,536, values. Professional audio, which uses 24 bit, has 2^{24}, or 16,777,216, values.

A single second of a 44.1 kHz, 16-bit **stereo** (meaning two channels) audio has:

- 44,100 samples per second
- each sample represented by 16 bits
- a total storage need per second of 44,100 samples / second × 16 bits / sample × 2 channels = 1,411,200 bits per second, or 176,400 bytes per second.



The blue continuous waveform represents an analogue signal, which is a smooth and continuous representation of sound. The digital signal consists of discrete samples taken at regular intervals (sampling rate), illustrating how the continuous analogue signal is converted into a series of discrete points in digital form.

Storage formats

There are many different types of file formats for storing audio. The most common are WAV, AIFF, MP3 and FLAC. They mainly differ by whether they are compressed or uncompressed. Uncompressed formats store the raw binary data, whereas compressed formats use algorithms to reduce the file size for storage or transmission. Just like with image compression, audio

◆ **Stereo:** a method of sound reproduction that uses two or more audio channels to create the perception of sound coming from different directions, enhancing the sense of spatial depth and dimension.

compression attempts to reduce the file size by removing parts of the audio signal that are less noticeable to human senses, in this case the ears. There are both lossy and lossless types of compression used with audio. Lossless algorithms compress the data without any loss of quality, whereas lossy algorithms permanently remove audio that is less noticeable to the human ear on the recording.

- **WAV** (Waveform Audio File Format): uncompressed
- **AIFF** (Audio Interchange File Format): uncompressed
- **MP3** (MPEG Audio Layer III): compressed (lossy)
- **FLAC** (Free Lossless Audio Codec): compressed (lossless)

REVIEW QUESTIONS

1. What is the main difference between an analogue signal and a digital signal in the context of audio?
2. What is the process of converting an analogue audio signal into a digital signal called, and what does it involve?
3. Calculate the storage needed per minute for a 44.1 kHz, 16-bit stereo audio file.
4. Explain the difference between lossy and lossless audio compression and give an example of each type of format.

PROGRAMMING EXERCISE

Explore audio files further using the code below. This will allow you to analyse the amplitude of any MP3 file.
You will need to install the following libraries:

- soundfile
- numpy
- matplotlib
- scipy.

Run this command in your terminal:

```
pip install soundfile numpy matplotlib scipy
```

Python

```python
import soundfile as sf
import numpy as np
import matplotlib.pyplot as plt
from scipy.fftpack import fft
# Load the audio file
samples, sample_rate = sf.read("name_of_file.mp3")
# If stereo, select one channel
if samples.ndim > 1:
    samples = samples[:, 0]
# Visualize the waveform
plt.figure(figsize=(12, 6))
plt.plot(samples)
plt.title("Audio Waveform")
```

```
plt.xlabel("Sample Index")
plt.ylabel("Amplitude")
plt.show()
# Perform FFT
spectrum = fft(samples)
frequencies = np.fft.fftfreq(len(spectrum), 1 / sample_rate)
plt.figure(figsize=(12, 6))
plt.plot(frequencies[:len(frequencies)//2],
np.abs(spectrum[:len(spectrum)//2]))
plt.title("Audio Spectrum")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.show()
```

Video

Videos are made up of various components that are all contained within an encapsulated container format such as MP4, MKV or AVI. The components are:

Digital video playback is similar to a flipbook: a number of images that are shown quickly, creating the illusion of motion

- frames (visual data)
- audio tracks
- metadata
- subtitles and closed captions.

Audio is stored as described in the Audio section above, and metadata and subtitles are stored as text, so this section will focus only on how the video data is stored.

Video is essentially stored as a sequence of still images, otherwise known as "frames". When played in quick succession (usually 24 to 60 frames per second), these frames create the illusion of motion. This is very similar to the technique you may have used to create a flipbook. The frames are stored and encoded in binary format, utilizing various techniques to optimize space and ensure efficient playback.

Frames

In their raw form, frames are stored the same as images, with each pixel having a value that can be represented using a colour model such as RGB. To improve colour efficiency, frames are often converted from the RGB colour model to a different one such as YUV. This helps with compression, as this colour model emphasizes luminance (brightness), which the human eye is more sensitive to than changes in colour detail.

However, we cannot store frames in the same way as we store photos because, in this format, they would be too large. They need to be compressed, and there are two main techniques used for this: spatial (intraframe) and temporal (interframe).

Compression techniques

Spatial compression is particularly effective and commonly used for video that has significant detail variation in each frame. It reduces file size by eliminating redundant information within each frame, such as colour depth or detail levels. This approach is important for videos with a lot of detail that may change significantly between frames, such as animations, nature documentaries and live news broadcasts.

Temporal compression is particularly effective and commonly used for video that has consistent motion across frames. It reduces file size by eliminating redundant information between consecutive frames, capturing only the changes or movements from one frame to the next. As a predictive compression technique, it predicts frame content based on the preceding and sometimes following frames, only storing the differences. This approach is important for videos with a lot of detail that may change significantly between frames, such as animations, nature documentaries and live news broadcasts.

REVIEW QUESTIONS

1. What is the role of frames in a video, and how do they create the illusion of motion?
2. Explain the difference between spatial compression and temporal compression in video storage.
3. Describe how converting video frames from the RGB colour model to the YUV colour model can improve compression efficiency.
4. Calculate the total storage needed for a 10-minute video with a frame rate of 30 frames per second, using 24-bit colour depth and a resolution of 1920×1080 pixels. Assume no compression.

Different binary methods for storing integers

Unsigned binary

This is the system we covered in Section A1.2.1. This system only represents positive integers using straightforward binary digits (0s and 1s).

Signed binary

This includes methods for representing both positive and negative integers.

Two's complement:

Two's complement is a method for representing signed integers in binary, where the most significant bit (MSB) indicates the sign (0 for positive, 1 for negative). To convert a positive binary number to its negative counterpart in two's complement, you first invert all the bits (change 0s to 1s and 1s to 0s) and then add 1 to the least significant bit (LSB).

For example:

00000101 = +5

Invert the bits

11111010

Add 1

11111011 = –5

However, a limitation of two's complement is that, in an 8-bit system, it reduces the range of representable numbers. Instead of being able to represent 0 to 255, as with unsigned binary, two's complement allows for numbers ranging from –128 to +127, effectively halving the number of positive values that can be represented.

One's complement:

One's complement is a binary representation method for signed integers, where the most significant bit (MSB) indicates the sign (0 for positive, 1 for negative). To obtain the one's complement of a positive number, you simply invert all the bits (change 0s to 1s and 1s to 0s).

For example:

00000101 = +5

11111010 = –5

Unlike two's complement, one's complement has two representations of the number zero: positive zero (00000000) and negative zero (11111111). This is one of its main limitations and can cause confusion when using arithmetic and logical operations. This means two's complement is normally the preferred system to use. Similarly to two's complement, one's complement also has a limited range, representing numbers from –127 to +127.

Sign-magnitude:

Sign-magnitude is a binary representation method for signed integers where the most significant bit (MSB) serves as the sign indicator, with 0 representing positive numbers and 1 representing negative numbers. The remaining bits represent the magnitude of the number, like how unsigned binary numbers work.

For example:

00000101 = +5

10000101 = –5

This system also has two representations for zero: positive zero (00000000) and negative zero (10000000). It also has the same range as one's complement, from –127 to +127. It is a simple system but, like one's complement, is less efficient compared to two's complement.

Binary-coded decimal

Binary-coded decimal (BCD) is a method of representing decimal numbers where each digit of the decimal number is encoded separately into its own binary form. Unlike pure binary representation, which converts the entire decimal number into a single binary sequence, BCD assigns a 4-bit binary code to each decimal digit (0–9).

For example:

0100 0101 = 45

as 0100 represents 4, and 0101 represents 5

This system is useful where exact decimal representation is crucial, such as financial applications or digital clocks, as it avoids the rounding errors that can occur in other systems. However, due to using four bits per digit, more bits are required to store numbers, making it less space efficient than pure binary representations. Calculations using BCD are also more complex as they require additional steps to handle carry and overflow, so they are not good choices for general-purpose computing.

Gray code (reflected binary code)

Gray code is a binary system where two successive values are only allowed to differ by one bit. That makes this system particularly useful in situations where data integrity during transitions is important. An example system is a robotic arm where we want to monitor its position. As the arm rotates, the rotary encoder generates a sequence of binary outputs corresponding to the arm's angle. If the encoder used standard binary code, small mechanical vibrations or inaccuracies could cause multiple bits to change simultaneously, leading to incorrect readings. However, by using Gray code, the risk of these transition errors is minimized.

■ Comparison of Gray code to standard binary for the numbers 0–7

Numbers	Standard binary	Gray code
0	000	000
1	001	001
2	010	011
3	011	010
4	100	110
5	101	111
6	110	101
7	111	100

Excess-N (biased representation)

Excess-N is a system where a fixed bias (N) is added to the actual value to form an encoded value, and you subtract this bias to decode it. This is used to make all signed integers appear as non-negative binary numbers to allow for easier comparisons and arithmetic operations.

For example, with Excess-3:

The decimal number 2 would be encoded as:

2 + 3 = 5

0101

The decimal number –2 would be encoded as:

–2 + 3 = 1

0001

In an 8-bit system, Excess-127 is often used, which adds 127 to encode an 8-bit number and subtracts 127 to decode it. If you consider trying to order a set of signed binary numbers, this can be difficult as the negative numbers are larger binary numbers than the positive.

For example, take 127 and –127 (using sign-magnitude):

Pre-encoded numbers:

01111111 = 127

10000001 = –127

When we encode these numbers with Excess-127, the positive numbers now appear larger than the negative numbers, making them easier to put in order:

Encoded numbers (Excess-127):

127 + 127 = 254

11111110

–127 + 127 = 0

00000000

After this process has been completed, we decode the numbers again to return them to their original form:

Decoded numbers (Excess-127):

254 – 127 = 127

11111110

0 – 127 = –127

00000000

Fixed-point representation

Fixed-point representation is a method used to store real numbers (numbers with fractional parts) in binary by fixing the position of the binary point. In a fixed-point system, the binary point is placed at a predetermined position, either between certain bits or at a specific location in the binary sequence. This allows for a straightforward representation of fractional numbers, though with some trade-offs in terms of precision and range.

For example, if we want to represent 5.25 in an 8-bit system where four bits represent the integer and four bits represent the fractional part:

Integer part (four bits): 0101 = 5

Fractional part (four bits): 0100 = 0.25

Combined: 0101.0100

Note: Binary fractions are used for the fractional part, where the first bit to the right represents $\frac{1}{2}$, the second bit represents $\frac{1}{4}$, the third $\frac{1}{8}$ and the fourth $\frac{1}{16}$. So, in the example above, we have 0 $\frac{1}{2}$, 1 $\frac{1}{4}$, 0 $\frac{1}{8}$ and 0 $\frac{1}{16}$.

The number of bits assigned in this system limits the range and precision. In this example, with a 4-bit signed integer, we only have the range of –8 to 7.9375, with the smallest representable value being 0.0625 ($\frac{1}{16}$). This means this system is unable to handle very large or very small numbers effectively. However, it is a simpler and faster system compared to floating-point arithmetic (see below), and does not require any complex operations to adjust the position of the binary point.

Floating-point representation

Floating-point representation is a method used to represent real numbers that can have a very large range or fractional parts. It does this by storing numbers in a format that includes a sign, an exponent and a mantissa (or significand). This format allows computers to efficiently handle very large numbers, very small numbers and numbers with fractional parts, all with a reasonable degree of precision.

Using the IEEE 754 standard for single-precision floating-point numbers (which uses 32 bits):

1. Sign bit (one bit):
   The sign bit determines whether the number is positive or negative.
2. Exponent (eight bits):
   This is used to scale the number by a power of two and is stored using the Excess-127 system in its "biased" form; in other words, 127 is added to the actual exponent value.
3. Mantissa (23 bits):
   This represents the significant digits of the number. The mantissa does not store leading ones (in normalized form).

For example, this is how we could represent the decimal number –5.75:

Convert the number to binary:

101.11

Normalize this number to the form of $1.xxxxx \times 2^n$:

1.0111×2^2

Determine the components:

Sign bit: 1 (as it is a negative number)

Exponent: 2 + 127 (Excess-127) = 129 = 10000001

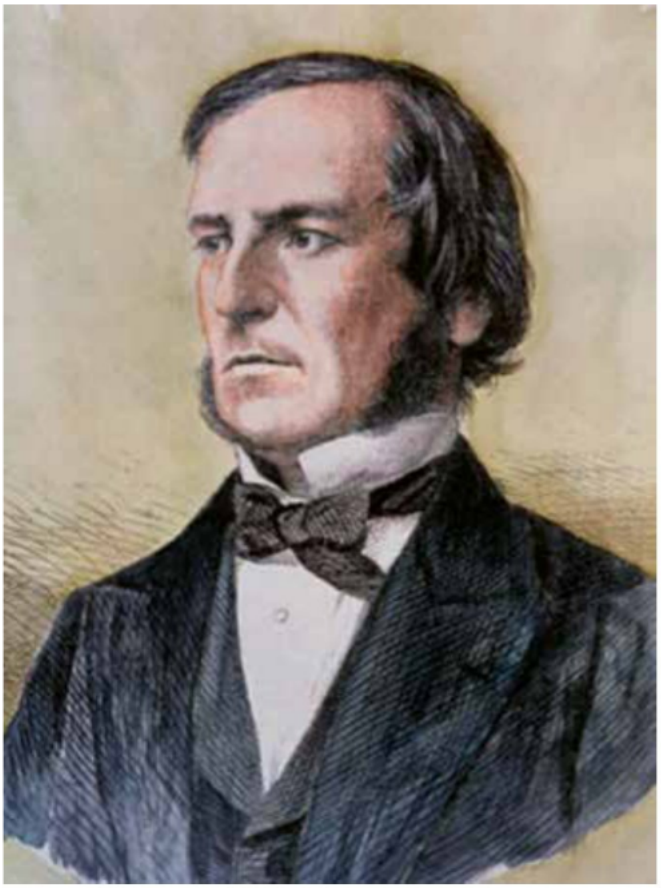
Mantissa: 01110000000000000000 (ignoring the leading 1)

So, the IEEE 754 single-precision binary representation of –5.75 is:

1 (sign) 10000001 (exponent) 01110000000000000000000 (mantissa)

This system allows for the representation of both very large and very small numbers, which is essential in scientific computing, engineering and graphics, and is more precise than fixed-point representation.

However, it is still not precise enough to represent all decimal numbers, and this can lead to rounding errors. The complexity of the system also makes it slower than others, often requiring special handling in hardware.



George Boole, the British mathematician who developed Boolean algebra, laying the groundwork for digital logic and modern computer science

A1.2.3 Purpose and use of logic gates

The history of logic gates

In the mid-19th century, a British mathematician named George Boole developed an algebraic system known as "Boolean algebra". This provided a mathematical framework for representing logical statements and operations that laid the foundations for modern digital logic.

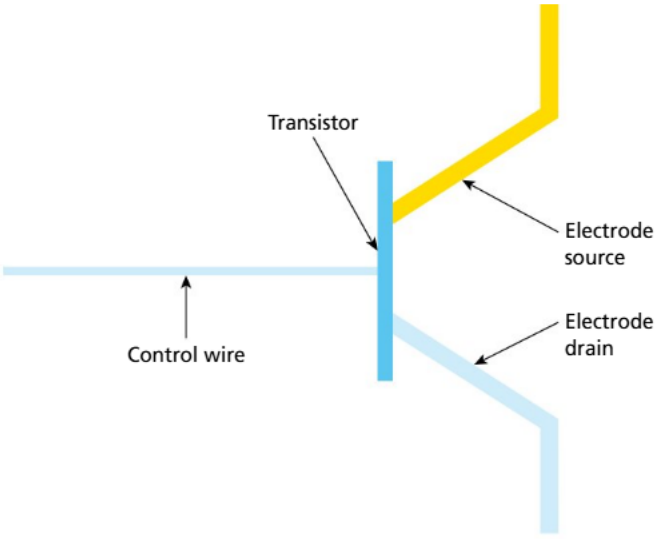
In the early 20th century, an American mathematician and electrical engineer named Claude Shannon was the first person to recognize the potential of Boolean algebra for electrical circuit design. He demonstrated how the design of electrical relay circuits could be optimized using Boolean algebra, and then the development of semiconductor technology further propelled the evolution of logic gates.

The transistor was then invented in 1947 at Bell Labs, which made it possible to build compact and efficient logic gates. By the 1960s to 1970s, integrated circuits were incorporating multiple transistors on a single chip, which led to the development of microprocessors. Logic gates are the building blocks of modern digital systems, from the basic calculator to advanced supercomputers.

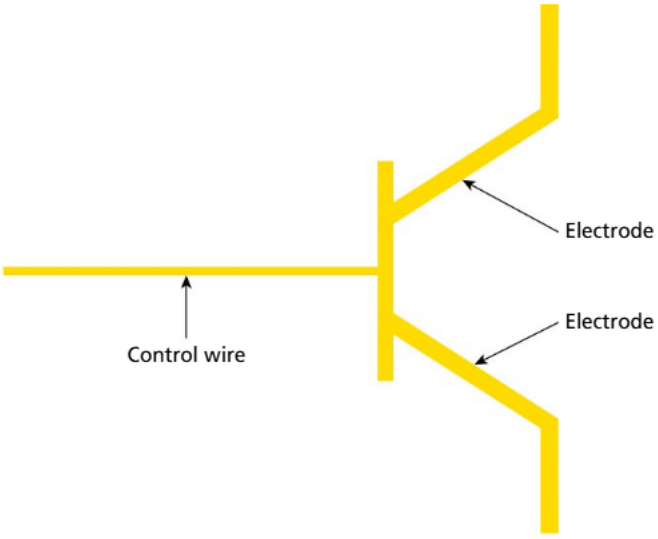
Basic gates

Logic gates are fundamental components in digital electronics, crucial for building various types of circuits within computers and other digital devices. The basic types of logic gates include AND, OR and NOT gates, each performing a specific logical function. These gates take one or more binary inputs and produce a binary output based on the logical operation they perform. To understand and verify the behaviour of these gates, we use truth tables, which systematically list all possible input combinations and their corresponding outputs, providing a clear representation of the gate's function. Additionally, each gate has a corresponding Boolean algebra representation that simplifies complex logical expressions.

Logic gates are made up of transistors, which act as electronic switches, allowing or blocking the flow of electrical current. In a transistor, the control wire (or "gate") regulates the current between the two electrodes, known as the "source" and the "drain". When voltage is applied to the gate, it allows current to flow from the source to the drain, enabling the transistor to switch states and perform logical operations.



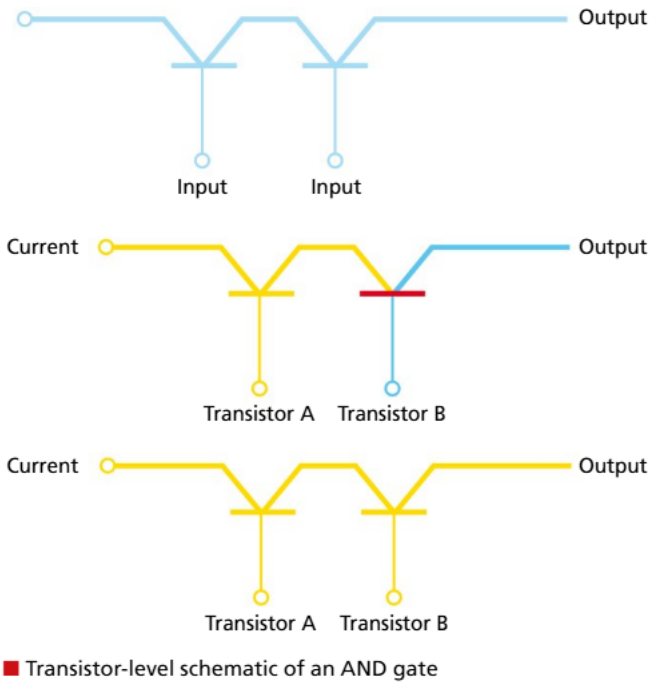
A Buffer gate that shows the inner workings of the gate – when the control wire is off, no electricity can flow between the electrodes



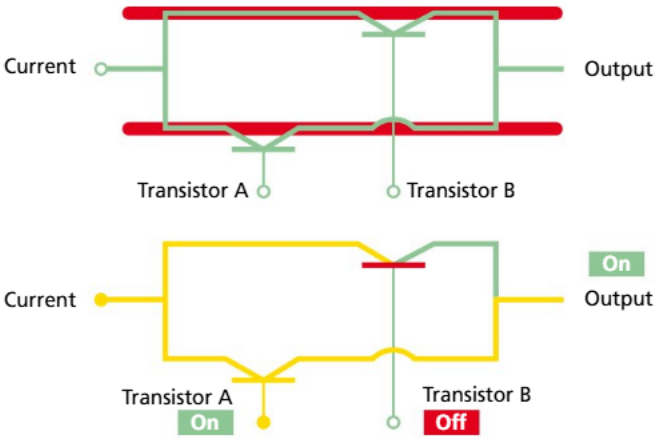
When electricity flows down the control wire, the transistor allows for the flow of electricity between electrodes

Above is a simple Buffer gate, where we can consider the control wire as the input and the electrode drain as the output. If the input is on, the output is on; and if the input is off, the output is off. We can show this using a truth table:

Buffer gate truth table	
Input A	**Output X**
1	1
0	0



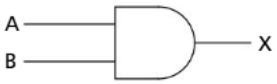
■ Transistor-level schematic of an AND gate



■ Transistor-level schematic of an OR gate

AND gate

The diagram on the left shows an AND gate implemented using transistors. The gate has two inputs and one output. If only one of the inputs is on, one of the transistors without an input would stop the current passing through. It is only when both inputs are high (1) that the transistors allow current to pass through, resulting in a high output (1).



■ AND gate

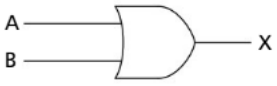
■ **Input and output rules:** The AND gate outputs 1 only if both inputs are 1

AND gate truth table		
Input A	**Input B**	**Output X**
0	0	0
0	1	0
1	0	0
1	1	1

Boolean algebra: $X = A \cdot B$

OR gate

The diagram on the left shows an OR gate implemented using transistors. The gate has two inputs – A and B – and one output. When either input A or B is high (1), the corresponding transistor turns on, allowing current to flow through the circuit and resulting in a high output (1). When both inputs are low (0), neither transistor conducts and the output remains low (0).

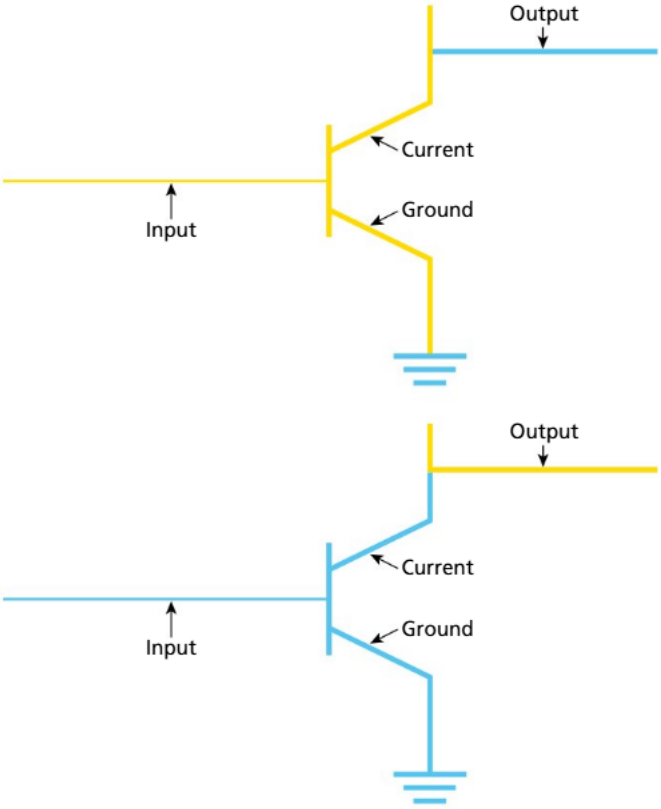


■ OR gate

■ **Input and output rules:** The OR gate outputs 1 if at least one input is 1

OR gate truth table		
Input A	**Input B**	**Output X**
0	0	0
0	1	1
1	0	1
1	1	1

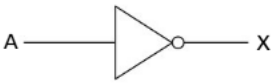
Boolean algebra: $X = A + B$



■ Transistor-level schematic of a NOT gate

NOT gate

The diagram below illustrates a NOT gate (inverter) using a transistor. In this configuration, the input line is connected to the gate of the transistor, the output line is connected to the source and the drain is connected to ground.



■ NOT gate

When the input is high (1), the transistor turns on, allowing current to flow from the source to the drain, effectively grounding the output and resulting in a low voltage at the output (0). When the input is low (0), the transistor turns off, preventing current from flowing to the ground. In this state, the output then outputs a high voltage (1).

■ **Input and output rules:** The NOT gate outputs the opposite value of the input

NOT gate truth table	
Input A	**Output X**
0	1
1	0

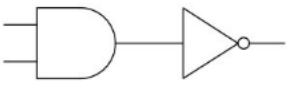
Boolean algebra: $X = \overline{A}$

Derived (complex) gates

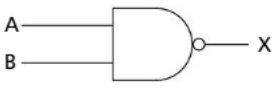
The following gates – NAND, NOR, XOR and XNOR – are examples of derived gates. Derived gates are combinations of the basic gates and provide more complex logic functions. To show these, we will move up a level of abstraction and, rather than examine the transistor schematic, we will look at how the basic gates are combined to create them.

NAND gate (NOT AND)

A NAND gate is constructed with an AND gate followed by a NOT gate. Due to this, it gives the opposite output to an AND gate. If both inputs are high (1), the output is low (0). In all other cases, the output is high (1).



■ How a NAND gate is constructed: an AND gate followed by a NOT gate



■ NAND gate

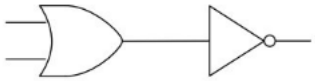
■ **Input and output rules:** The NAND gate outputs 1 unless both inputs are 1

NAND gate truth table		
Input A	**Input B**	**Output X**
0	0	1
0	1	1
1	0	1
1	1	0

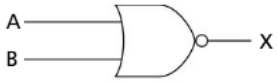
Boolean algebra: $X = \overline{A \cdot B}$

NOR gate (NOT OR)

An NOR gate gives the opposite output to an OR gate as it is constructed using an OR gate followed by a NOT gate. This means that it is only when both inputs are low (0) that the output is high (1). In all other cases, the output is low (0).



How a NOR gate is constructed: an OR gate followed by a NOT gate



A NOR gate

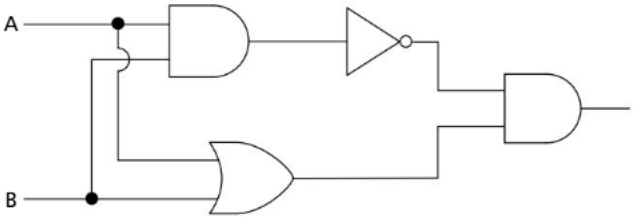
Input and output rules: The NOR gate outputs 1 only if both inputs are 0

NOR gate truth table		
Input A	**Input B**	**Output X**
0	0	1
0	1	0
1	0	0
1	1	0

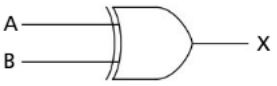
Boolean algebra: $X = \overline{A + B}$

XOR gate (exclusive OR)

The XOR (exclusive OR) gate differs from the OR gate in one key way: its output is true only when the inputs are different. This means the XOR gate outputs true when exactly one of the inputs is true, but false when both inputs are the same. For example, if both inputs are high (1), the XOR gate's output is low (0). This is unlike the OR gate, which would output high (1) in this case.



How an XOR gate is constructed



An XOR gate

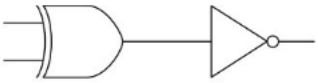
Input and output rules: The XOR gate outputs 1 if the inputs are different

XOR gate truth table		
Input A	**Input B**	**Output X**
0	0	0
0	1	1
1	0	1
1	1	0

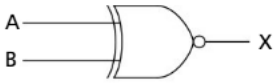
Boolean algebra: $X = A \oplus B$

XNOR gate (exclusive NOT OR)

The XNOR gate is constructed using an XOR gate followed by a NOT gate. This means the output is the opposite to an XOR gate, only outputting high (1) when both inputs are the same (either high or low).



How an XNOR gate is constructed: an XOR gate followed by a NOT gate



An XNOR gate

Input and output rules: The XNOR gate outputs 1 if the inputs are the same

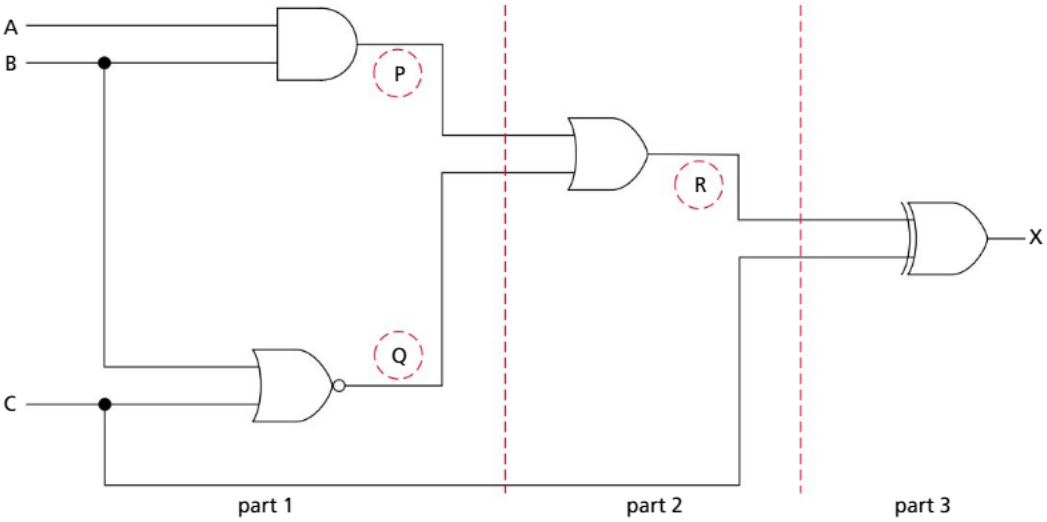
XNOR gate truth table		
Input A	**Input B**	**Output X**
0	0	1
0	1	0
1	0	0
1	1	1

Boolean algebra: $X = \overline{A \oplus B}$

A1.2.4 Constructing and analysing truth tables

Truth tables to predict the output of simple logic circuits

The following diagram shows a logic circuit, where a number of logic gates are connected together. In this scenario, you need to be able to handle circuits with up to three inputs.



When creating a truth table, first enter the three inputs and their possible input states; in this case, A, B and C. As there are three inputs, you can calculate the number of rows you will need by 2^n, where n represents the number of inputs. In this example:

$2^3 = 8$

Populate the furthest right column, alternating between 0 and 1.

Populate the middle column by alternating, every two rows, between 0 and 1.

Populate the left-hand column by alternating, every four rows, between 0 and 1.

Following this pattern will give you every possible input state.

A	B	C
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

Once this is complete, we then add the intermediate values to make it easier to remember the state at each stage of the circuit. In this example, we have three intermediate values: P, Q and R. Finally, we add the output column, X.

A	B	C	P	Q	R	X
0	0	0				
0	0	1				
0	1	0				
0	1	1				
1	0	0				
1	0	1				
1	1	0				
1	1	1				

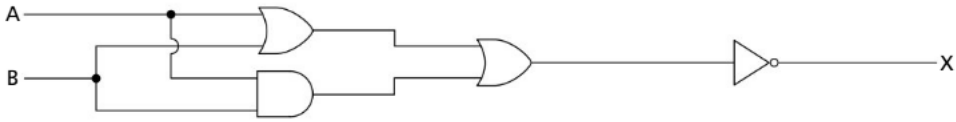
Now, starting with the intermediate values, work through the logic circuit.

A	B	C	P (A AND B)	Q (B NOR C)	R (P OR Q)	X (C XOR R)
0	0	0	0	1	1	1
0	0	1	0	0	0	1
0	1	0	0	0	0	0
0	1	1	0	0	0	1
1	0	0	0	1	1	1
1	0	1	0	0	0	1
1	1	0	1	0	1	1
1	1	1	1	0	1	0

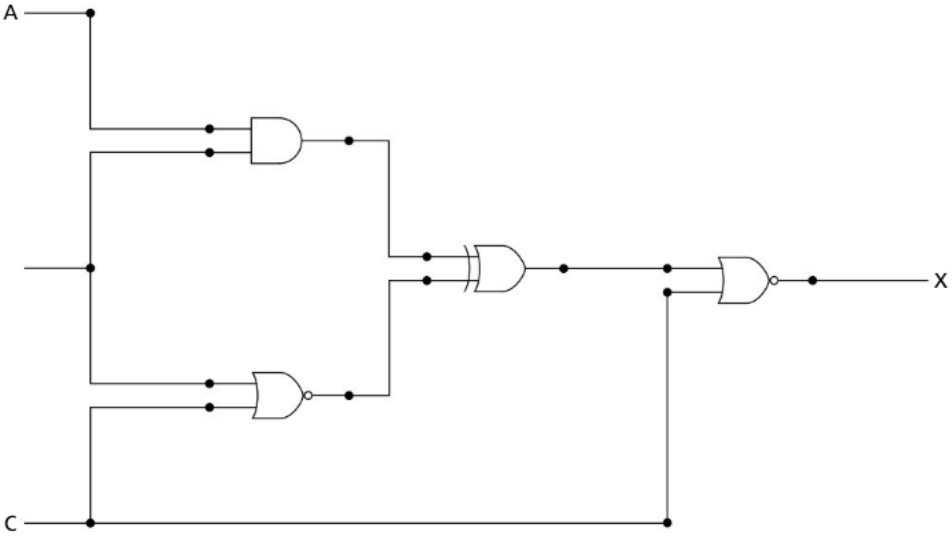
REVIEW QUESTIONS

Produce the truth tables for the following logic circuits.

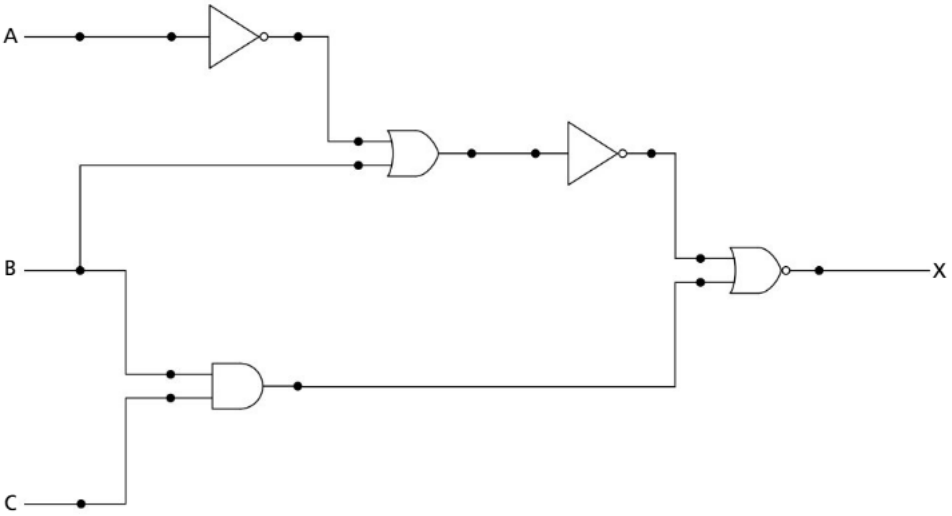
1



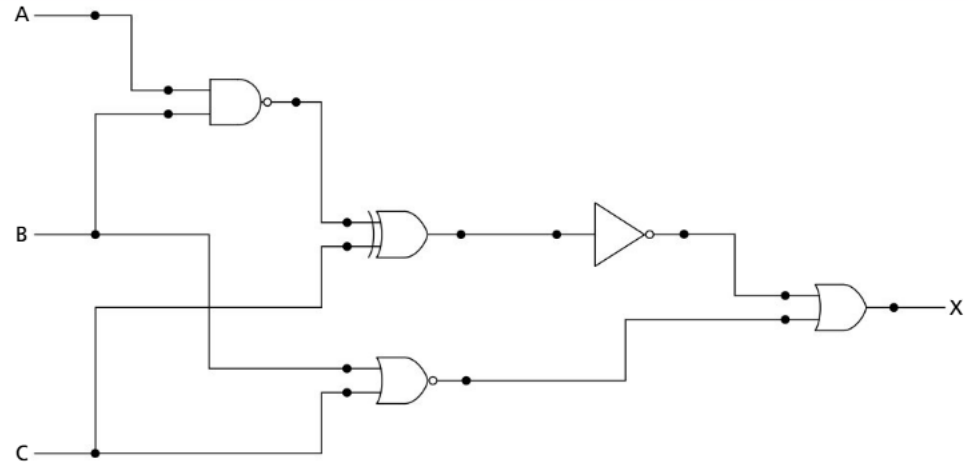
2



3



4



Truth tables to determine outputs from inputs for a problem description

Problem description: *A baby alarm that goes off when the alarm is switched on and the baby is crying or the room is too cold.*

With a problem description, we first need to identify the inputs. Here we have three: alarm switch, baby crying and room temperature, which we can represent as A, B and C and set up our truth table.

We know the alarm goes off if the device is switched on AND the baby is crying OR the room is too cold. From here, we can identify the logic gates in the description. We can determine from this that the device must be switched on before the alarm can go off, so if the switch is off, all outputs would be 0. If the device is switched on, at least one of the other two inputs must be on for the alarm to trigger. We have one intermediate value (baby crying OR room is cold), represented with P.

A (switch)	B (crying)	C (cold)	P (B or C)	X (A and P)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	0	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

REVIEW QUESTIONS

Complete the truth tables for the following problem descriptions:

1. A traffic light control system that changes to green only if the pedestrian button is not pressed and the road sensor detects no cars waiting on the side road.
2. A water irrigation system that activates if the soil is dry or the temperature is high, provided the system is manually enabled.
3. A nuclear missile launcher: the missile should only launch if the president and two of his three cabinet ministers flip the switch.

Logical expressions

We can represent logic circuits using Boolean algebra. If we use the baby alarm scenario again, we can represent this as:

$X = A \cdot (B + C)$

In Boolean algebra, parentheses are often used to indicate which operations should be performed first. However, there is a standard order of operations, similar to PEMDAS (or BODMAS) in mathematics. If no parentheses are present, follow this sequence:

- NOT
- AND (including NAND)
- OR (including NOR and XOR)

This ensures that operations are executed in the correct order for accurate results.

REVIEW QUESTIONS

Using the Boolean algebra from the gates in Section A1.2.3, write the Boolean algebra for the three logic circuits created in the previous problem descriptions:

1. Traffic light control system
2. Water irrigation system
3. Nuclear missile launcher

Karnaugh maps and algebraic simplification

Karnaugh maps (K-maps) are a tool that helps simplify Boolean expressions, making it easier to create simpler and more efficient digital circuits. Instead of using complicated algebraic methods, K-maps allow you to visually group terms from a truth table, which makes it faster to find a simplified expression. This process is useful because it reduces the number of logic gates needed in a circuit, saving time, space, cost and power consumption.

Two inputs

This two-input K-map is used for expressions with two variables. In this map, variable A is placed along the side and variable B across the top. However, the order of the variables doesn't matter – B could be along the side and A across the top. Both possible states (0 and 1) for each variable are shown in the map, representing all combinations of their values.

Expression: A + B

A \ B		B	B
		0	1
A	0		
A	1		

We split the expression at the OR operator and focus first on the term involving A. We populate the K-map where A is 1, which, in this case, corresponds to the entire bottom row. At this stage, we ignore B and only fill in the cells where A is 1.

A \ B		B	B
		0	1
A	0		
A	1	1	1

We now do the same for the second part of the expression: B.

A \ B		B	B
		0	1
A	0		1
A	1	1	1

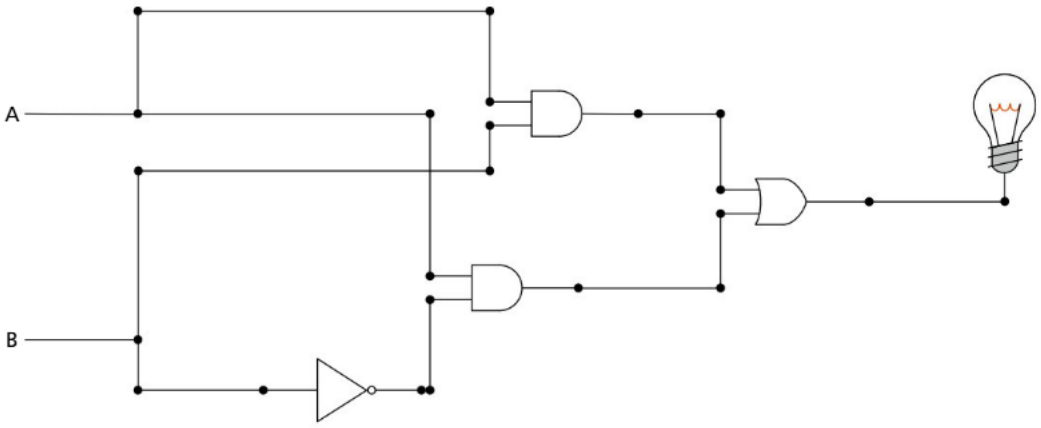
This completed K-map now shows the expression A + B.

This is another example of a completed map for the expression $\overline{A}$ + B:

A \ B		B	B
		0	1
A	0	1	1
A	1		1

Expression: $A \cdot B + A \cdot \overline{B}$

This is a more complicated expression, but still has only two inputs. Using this expression, the circuit would look like this:



The table map structure is still the same:

A \ B		B	B
		0	1
A	0		
A	1		

We again split the expression at the OR symbol, focusing initially on A · B. In this case, we insert a 1 into only one cell, where both A and B are 1.

A \ B		B	B
		0	1
A	0		
A	1		1

Next, we focus on the second part of the expression, $A \cdot \overline{B}$. In this case, we insert a 1 into the cell where A is 1 and B is 0.

A \ B		B	B
		0	1
A	0		
A	1	1	1

The K-map is now complete and shows that the value of B has no impact on the outcome, allowing us to simplify the expression to just A. If we create the circuit using this simplified expression, we can see that the circuit is significantly more efficient, while still performing the same function.

Three inputs

Expression: $\overline{A} \cdot B \cdot C + A \cdot \overline{B} \cdot C + A \cdot B \cdot C$

With three inputs, we use a similar K-map but, this time, we place two of the inputs across the top. The digits across the top may seem out of order compared to standard binary counting (00, 01, 10, 11). Instead, they follow the Gray code convention (see Section A1.2.2 for more information), where only one digit changes at a time. It is important to set the map up this way to ensure correct grouping and simplification.

C \ AB		AB	AB	AB	AB
		00	01	11	10
C	0				
C	1				

We now follow similar steps as with the two-input K-map. We separate the expression by the OR operator and focus on the first term: $\overline{A} \cdot B \cdot C$. In this step, we populate the K-map by inserting a 1 into the cells where A is 0, B is 1 and C is 1.

C \ AB		AB	AB	AB	AB
		00	01	11	10
C	0				
C	1		1		

Followed by: $A \cdot \overline{B} \cdot C$

C \ AB		AB	AB	AB	AB
		00	01	11	10
C	0				
C	1		1		1

And finally: $A \cdot B \cdot C$

C \ AB		AB	AB	AB	AB
		00	01	11	10
C	0				
C	1		1	1	1

Grouping the 1s and simplifying the expression

Although it wasn't explicitly stated before, you may have noticed the boxes drawn around groups of 1s in the K-maps. These boxes help simplify the Boolean expression, but there are some important rules that must be followed when grouping 1s:

- **Groups must contain powers of 2:** One, two, four, eight or sixteen 1s can be grouped together.
- **Groups must be rectangular or square:** Each group should form a rectangle or square shape.
- **Groups cannot be diagonal:** Adjacent 1s can only be grouped horizontally or vertically, not diagonally.
- **Groups must be as large as possible:** Always aim to make the largest groups to simplify the expression further.
- **Groups can overlap:** Some 1s may be included in more than one group if it helps form larger groups.
- **Minimize the total number of groups:** The goal is to use the smallest number of groups to cover all the 1s.

To determine the expression from the groups, we look at each group of cells and refer to the variables. If a variable's value stays the same across all the cells in the group, we keep that variable in the simplified expression. However, if the variable's value changes across the group, we discard it from the expression.

C \ AB		AB	AB	AB	AB
		00	01	11	10
C	0				
C	1		1	1	1

The first group is entirely along the bottom row, meaning C stays the same (C = 1), so we keep it in the expression. A changes from 0 to 1 between the cells in the group, so we discard A. B remains 1 in both cells, so we keep B. Therefore, the first part of our final expression is:

$B \cdot C$

C \ AB		AB 00	AB 01	AB 11	AB 10
C	0				
C	1		1	1	1

The second group, like the first, is located along the bottom row, meaning that C stays as 1 because it does not change across the group. In this case, A remains 1 in both cells, while B changes from 0 to 1. Since B changes, we discard B from this part of the expression. As a result, we keep A and C, giving us the second part of our expression:

$A \cdot C$

We then combine these expressions with an OR operator, giving us the final expression:

$B \cdot C + A \cdot C$

Common mistake

When setting up your K-map for three inputs, make sure to use Gray code for the headings, not standard binary. Gray code ensures that only one bit changes between adjacent cells, which helps when grouping 1s and simplifies the expression more effectively.

Wrapping around edges in K-maps

Here is the K-map for the expression:

$\overline{B} + A \cdot B \cdot C$

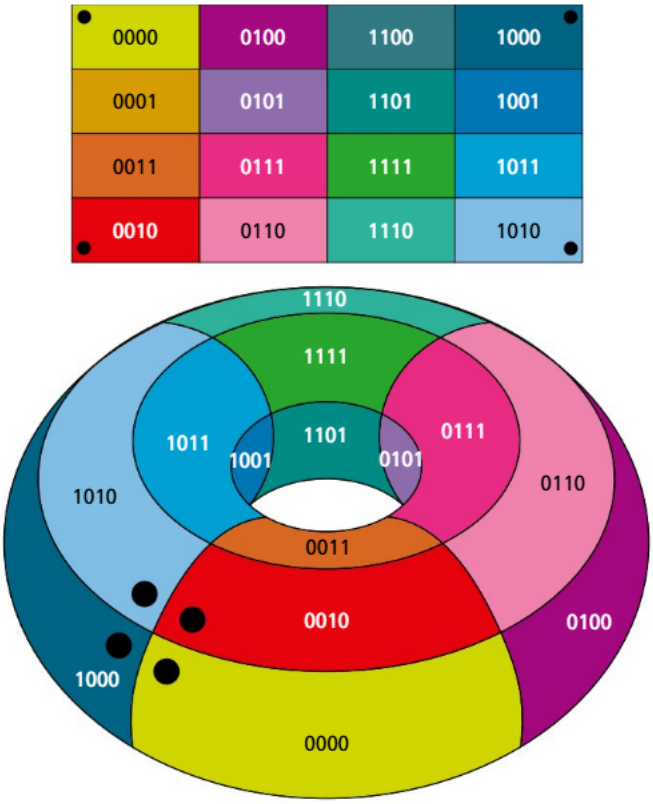
C \ AB		AB 00	AB 01	AB 11	AB 10
C	0	1			1
C	1	1		1	1

To group these 1s, you may assume this is the answer:

C \ AB		AB 00	AB 01	AB 11	AB 10
C	0	1			1
C	1	1		1	1

However, K-maps are considered three-dimensional, and groups can be formed from left to right and top to bottom (although only left to right is possible with three inputs). In this example, it is possible to build a larger group by combining the two groups on the edges, forming a square group of four 1s.

C \ AB		AB 00	AB 01	AB 11	AB 10
C	0	1			1
C	1	1		1	1



■ K-map drawn on a torus and in a plane – the dot-marked cells are adjacent

Using these groups, we can form the simplified expression:

$A \cdot C + \overline{B}$

REVIEW QUESTIONS

$\overline{A} \cdot B + A \cdot B$

1 Draw the truth table for the above expression.

2 Draw the Karnaugh map for the expression and write the simplified expression.

$\overline{A} \cdot B \cdot C + A \cdot B \cdot C + \overline{A} \cdot B \cdot \overline{C}$

3 Draw the truth table for the above expression.

4 Draw the Karnaugh map for the expression and write the simplified expression.

$\overline{A} \cdot \overline{B} \cdot C + \overline{A} \cdot B \cdot C + A \cdot \overline{B} \cdot C$

5 Draw the truth table for the above expression.

6 Draw the Karnaugh map for the expression and write the simplified expression.

A1.2.5 Constructing logic diagrams

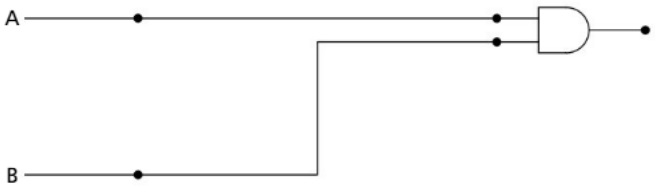
■ Designing digital circuits from Boolean algebra expressions

By understanding the principles of Boolean algebra, we can simplify complex logic expressions and translate them into circuit diagrams. This journey from abstract mathematical notation to circuit design is essential for creating efficient and reliable digital systems. We will start by creating the digital circuit from the expression below:

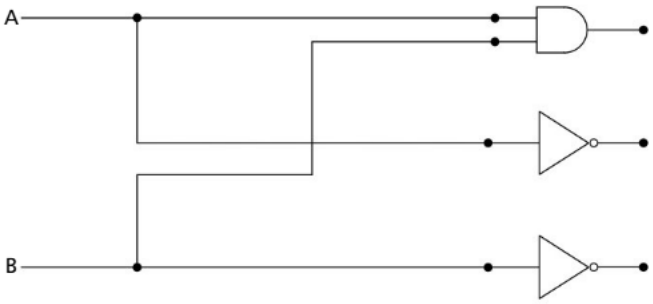
$Y = (A \cdot B) + (\overline{A} \cdot \overline{B})$

1 Start with two inputs: A and B.

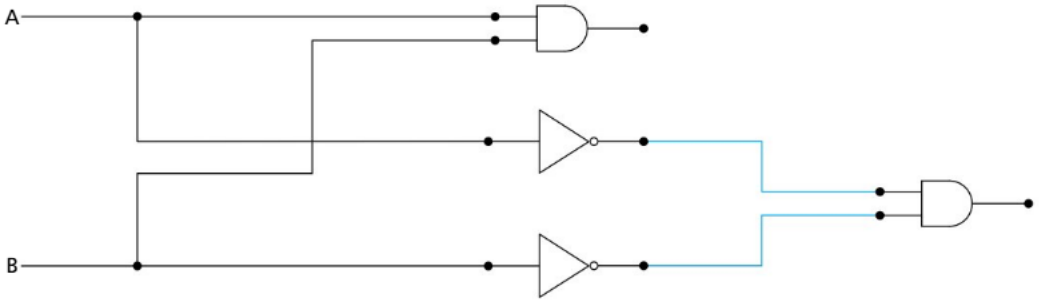
2 Work on the first parenthesis by introducing an AND gate and joining both A and B to it.



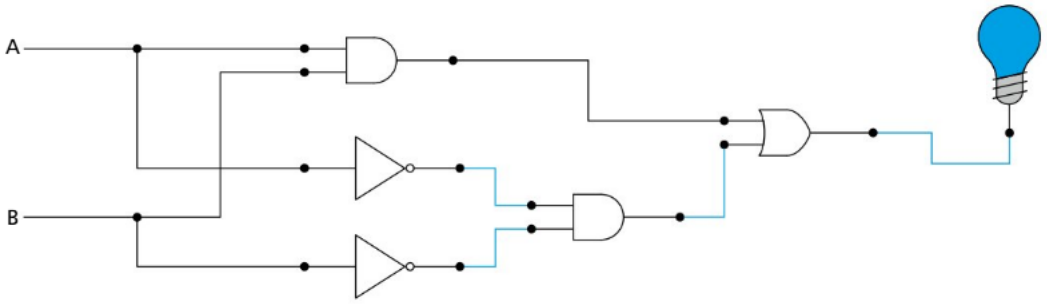
3 Work on the second parenthesis, connecting A to a NOT gate and B to a NOT gate.



4 Connect both the outputs to an AND gate.



5 Now work outside the parentheses and introduce the OR gate to link them together.



REVIEW QUESTIONS

Create the digital circuits from the following Boolean expressions.

1 $Y = (A \cdot B) \oplus \overline{A}$

2 $Y = (A + B) \cdot \overline{A \cdot B}$

3 $Y = A \cdot \overline{(B + C)}$

4 $Y = (A \oplus B) \cdot (B \oplus C)$

5 $Y = (A \cdot B) \cdot \overline{C} + \overline{(A + B)}$